

Restaurant Database Project

A database for a restaurant that handles reservations, orders, payments and stock, plus a data warehouse for reporting.

Code on GitHub: [Usf132/restaurant-db-data-warehouse](https://github.com/Usf132/restaurant-db-data-warehouse)

1. Overview

The project has two parts. The first is an **operational database** that runs the restaurant day to day. The second is a **data warehouse** that reorganises the same data so managers can analyse sales, payments, reservations and purchases.

14

tables

144

sample rows

2 + 3

functions and procedures

3

triggers

How a visit flows through the system

Reserve a table

>

Open an order

>

Add items

>

Stock goes down

>

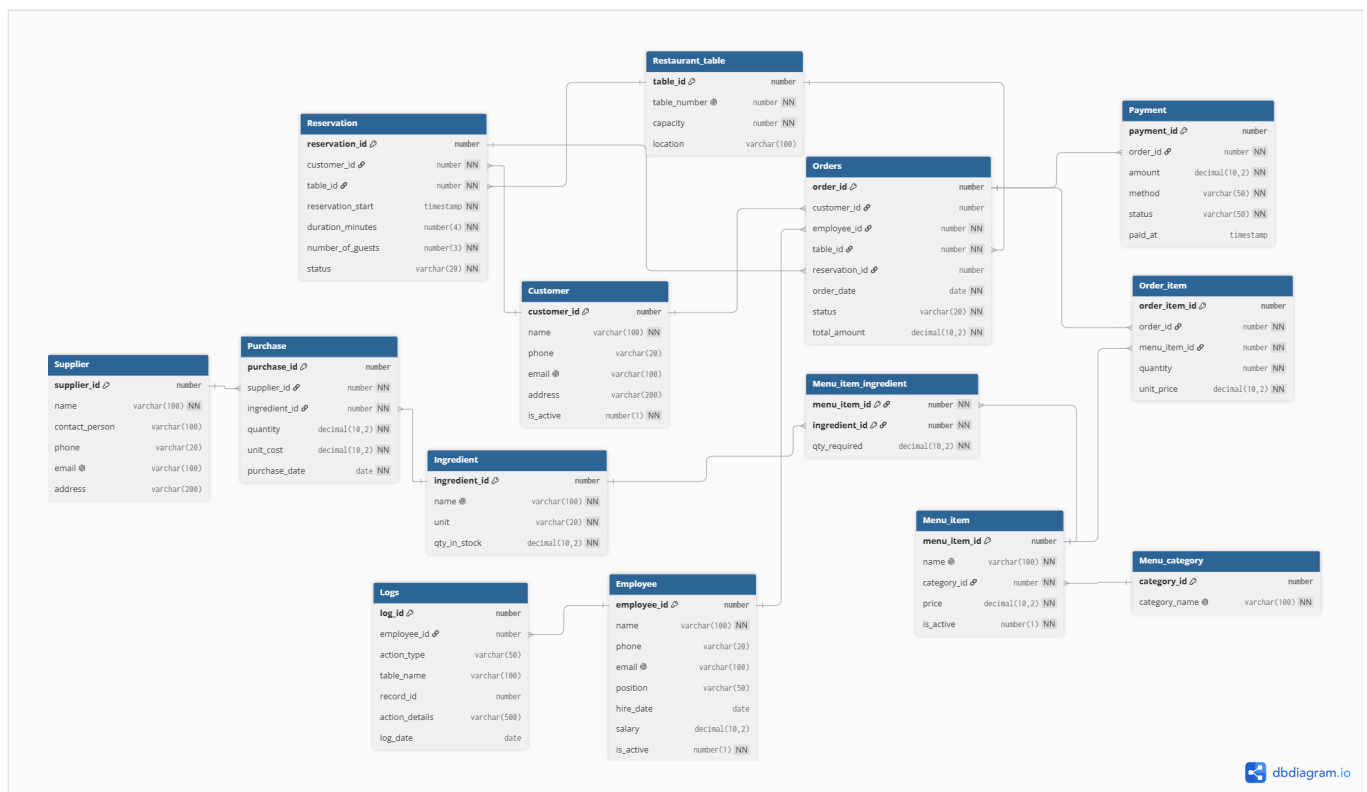
Customer pays

>

Order closes

2. The database design

The diagram shows the 14 tables and how they connect.



Entity relationship diagram (ERD).

Sales and service

Guests, tables, bookings, orders and payments.

Customer Restaurant_table Reservation Orders
Order_item Payment

Menu and kitchen

What is sold and what it is made of.

Menu_category Menu_item Menu_item_ingredient
Ingredient

Purchasing

Where the ingredients come from.

Supplier Purchase

Staff and audit

Who works here and what they changed.

Employee Logs

Rules built into the tables

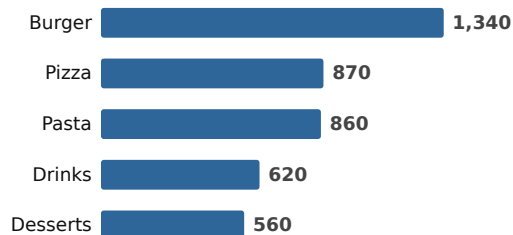
- Every table has a primary key, and emails, table numbers and names are unique.
- Check constraints keep data valid: prices above zero, stock never negative, statuses from a fixed list.
- A special unique index stops two confirmed bookings for the same table at the same start time.

3. What the sample data shows

We loaded 8 customers, 10 orders, 10 reservations and more. These charts come directly from that data.

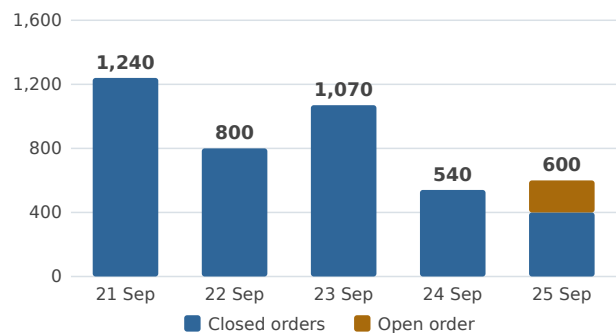
Revenue by category

Burgers earn the most (31.5% of 4,250).



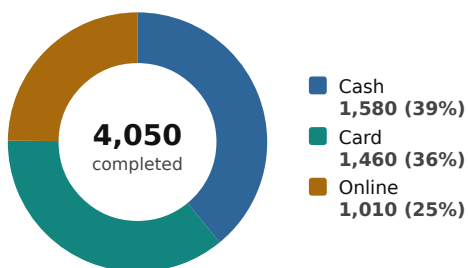
Revenue per day

Amber: one order still open.



Payment methods

Completed payments only.



Reservations by status

8 of 10 bookings are confirmed.



4. Business logic inside the database

Important rules live in the database itself, so they work no matter which application uses it.

Functions (2)

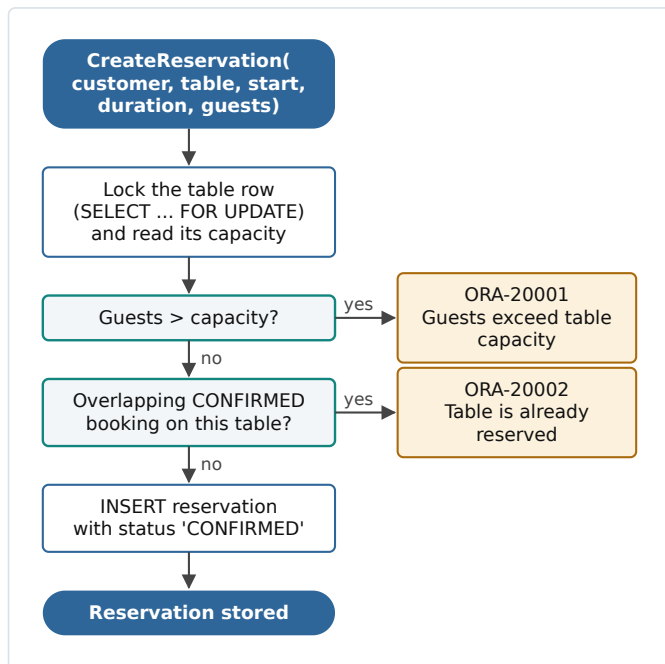
`CalculateOrderTotal` adds up an order's lines.
`GetCustomerVisitCount` counts a customer's orders.

Procedures (3)

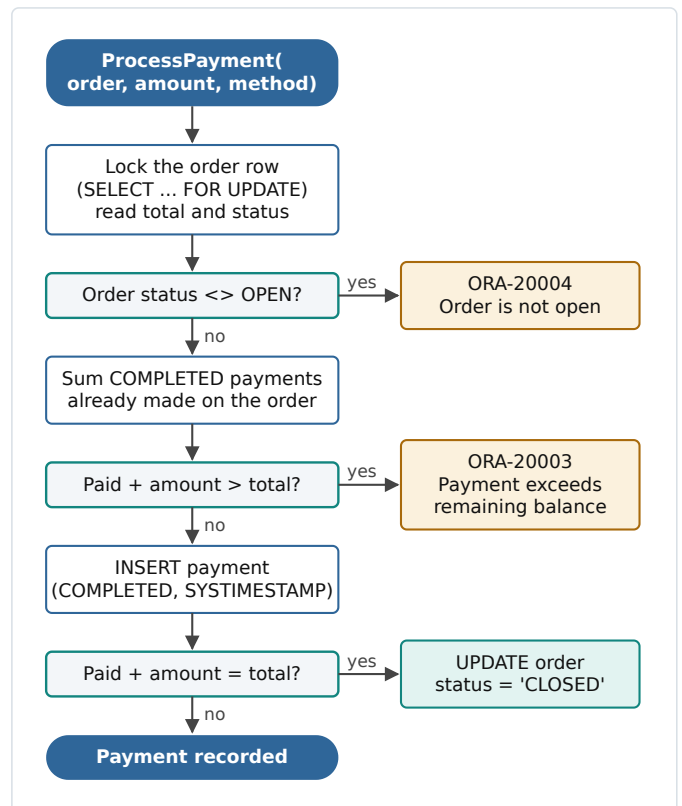
`GetCustomerOrders` lists a customer's orders.
`CreateReservation` books a table safely.
`ProcessPayment` records a payment.

Triggers (3): actions that happen automatically

`trg_inventory_update` : when an item is ordered, its ingredients are subtracted from stock.
`trg_price_audit` : when a menu price changes, the old and new price are written to the log.
`trg_order_total` : the order total is updated whenever its lines change.



How `CreateReservation` checks a booking.

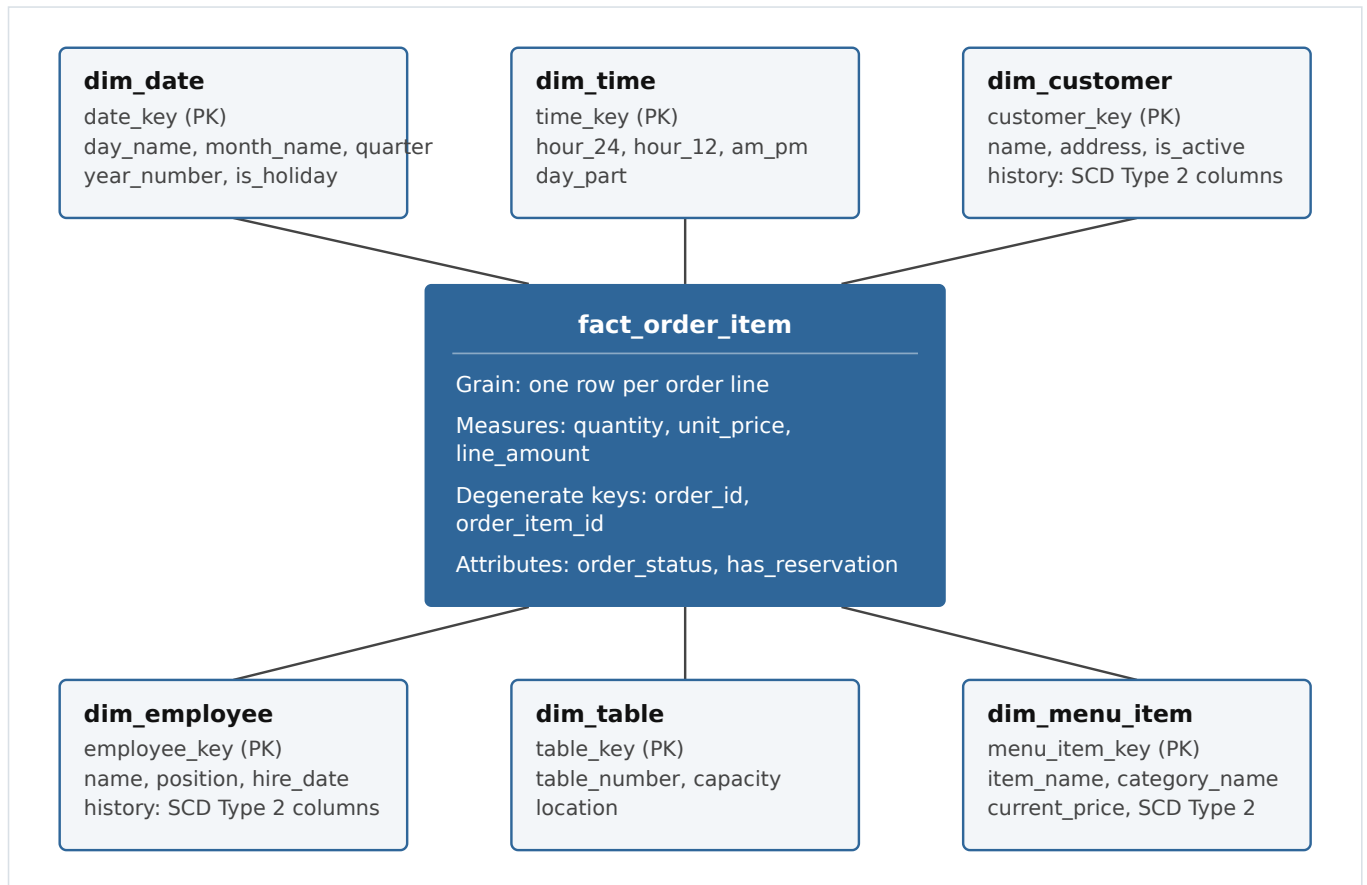


How `ProcessPayment` checks a payment.

Both procedures lock the row they are checking, so two people cannot book the same table or overpay the same order at the same moment.

5. The data warehouse

The warehouse uses a **star schema**: central *fact* tables hold events (a sale, a payment), and surrounding *dimension* tables describe them (when, who, what). Below is the star for sales lines.



Star schema for order lines. There are 5 fact tables and 9 dimensions in total.

Fact table	One row per	Question it answers
fact_order_item	order line	What sells, when, and who sold it?
fact_payment	payment	How do customers pay?
fact_reservation	reservation	How are tables booked or cancelled?
fact_purchase	purchase	How much do we spend per supplier?
fact_inventory_snapshot	ingredient per date	How does stock change over time?

History is preserved for customers, employees and menu items (SCD Type 2): when something like a price changes, a new row is added and the old one is kept, so past reports stay correct. Employee salary is deliberately left out of the warehouse for privacy.

6. Conclusion

The project covers the full path from a restaurant's daily operations to analysis: a validated database, automatic business rules through functions, procedures and triggers, and a star-schema warehouse ready for reporting.